Ders 10 Çalışma Özeti

Ders 10 Çalışma Özeti

Genel Konular

- Asıl anlatım yaklaşık 00:16'da başlar. Öncesi sohbettir.
- Datapath tekrarı
  - Register file, fonksiyon birimi (ALU + shifter), MUX B, MUX D, 16 bitlik kontrol kelimesi.
- Tek çevrimli bilgisayar (single-cycle computer)
  - Komut belleği (instruction memory), veri belleği (data memory), program sayacı (PC).
  - Üç adresleme modu ve 16 bitlik sabit uzunluklu komut formatı.
  - Komut tablosu (19 komut), bellek içeriğinden komut çözme örneği.
  - Kontrol birimi: komut çözücü (instruction decoder), hardwired kontrol.
  - Çevrim süresinin (clock period) hesabı.
- Boru hattı (pipeline) işleme
  - 3 aşamalı pipeline datapath (OF, EX, WB) ile hızlanma.
  - 4 aşamalı pipeline (IF, DOF, EX, WB) ve kontrol birimi.
  - Pipeline tehlikeleri (hazard): veri bağımlılığı (data hazard) ve dallanma (control/branch hazard).
- Giriş/Çıkış ve iletişim (Input/Output and Communications)
  - Çevre birimleri (peripherals): klavye, sabit disk, LCD ekran.
  - G/Ç arayüzü (I/O interface), port adresleme, chip select tasarımı.
  - Strobing ve handshaking.
  - Paralel ve seri iletişim; simplex, half duplex, full duplex.

Hocanın Özellikle Vurguladığı Kısımlar

- Datapath'in nasıl çalıştığını "gözünüz kapalı" bilmelisiniz.
  - Bu kısmı bilirseniz dersin yaklaşık %30'unu biliyorsunuz demektir.
- Fonksiyon birimindeki hiçbir blokta senkron eleman (flip-flop) yoktur.
  - Hepsi kombinasyonel devredir. Amaç bir clock periyodunda bir mikro işlem yapan tek çevrimli bilgisayardır.
- Kontrol kelimesi soruları sık ve kolay sorulardır.
  - Tabloları ezberlemeye gerek yok.
- Zero fill ile sign extend farklı işlemlerdir.
  - Zero fill: yüksek bitler 0 ile doldurulur (immediate operand).
  - Sign extend: yüksek bitler işaret bitiyle doldurulur (dallanma adresi).
- CPU = kontrol birimi + datapath. "Yıllar sonra aklınızda kalacak temel konu budur."
- Pipeline sınav sorusu tipi:
  - "15 mikro işlem, 5 aşama verildiğinde geleneksel yöntemle pipeline'ı karşılaştırın" gibi.
  - Toplam çevrim sayısı = (aşama sayısı − 1) + mikro işlem sayısı.
- Pipeline tehlikelerinde amaç problemi görmek ve bir çözüm gerektiğini anlamaktır.
  - Ayrıntıları Bilgisayar Mimarisi dersinde görülecek.
- G/Ç arayüzü bölümü en önemli konudur. Dikkatli dinleyin.

Kısa Tekrar Notları

- Komut formatı: 16 bit. Opcode: en yüksek 7 bit (15-9). Kalan 9 bit: üç tane 3 bitlik alan (DR, SA, SB).
- Adresleme modları: register, immediate, jump and branch.
- Opcode hem işlemi hem adresleme modunu gösterir.
- Immediate operand 3 bittir (0-7). Kullanılmadan önce 16 bite zero fill ile genişletilir.
- Dallanma adresi: adres left ve adres right alanları birleştirilir (concatenate) → 6 bit işaretli sayı → sign extend. SA test edilen register'ı gösterir.
- BRZ: $R[SA] = 0$ ise $PC \leftarrow PC +$ se AD. BRN: $R[SA] < 0$ ise aynı.
- Dallanma ve atlama dışındaki komutlarda ayrıca $PC \leftarrow PC + 1$ yapılır. Koşulu sağlanmayan dallanmada da PC 1 artar.
- Ek kontrol işaretleri: MW (memory write), PL (PC load), JB (0: branch, 1: jump), BC (0: sıfırsa, 1: negatifse dallan).
- Komut çözücü girişleri: opcode bitleri 15, 14, 13 ve 9. Örnek: $MB = I_{15}$, $RW = \overline{I_{14}}$.
- Tek çevrimli bilgisayar kritik yolu: $0.2 + 4 + 0.6 + 0.2 + 4 + 0.2 + 0.6 = 9.8$ ns.
- Pipeline hızlanması (3 aşama, 7 mikro işlem): $7 \times 2.4 = 16.8$ ns → $9 \times 1 = 9$ ns.
- Pipeline (4 aşama, 7 komut): $7 \times 3.4 = 23.8$ ns → $10 \times 1 = 10$ ns, yaklaşık 2.4 kat hızlı.
- Veri bağımlılığı: yazılım çözümü NOP eklemek, donanım çözümü bekletme (stall) ya da sonucu doğrudan iletmek.
- Dallanma komutundan sonra 2 NOP eklenir.
- G/Ç arayüzü: Port A, Port B, Control ve Status register'ları. CS = 0 ise veri yolu yüksek empedans (high-Z).
- Strobe tek tel: karşı tarafın veriyi aldığı bilinmez. Handshaking iki tel (request/reply): güvenli.

Detaylı Açıklamalar

1. Tek çevrimli bilgisayarın komut seti mimarisi

- Bileşenler: komut belleği, veri belleği, PC, register file, datapath, kontrol birimi.
- **Adresleme modları:**
  - **Register:** iki kaynak register (SA, SB) ve bir hedef register (DR). Örnek: $R5 \leftarrow R1 + R3$.
  - **Immediate:** komutun içinde bir sabit (operand) vardır. Örnek: $R3 \leftarrow R2 + 3$.
    - Sabit için yalnızca 3 bit vardır. Bu yüzden 0-7 arasında değer alabilir.
    - Sebep: sabit uzunluklu komut formatı (fixed length instruction). 8086'da ise komut uzunlukları değişkendir.
  - **Jump and branch:** koşullu veya koşulsuz dallanma. Koşul SA register'ının içeriğine göre test edilir.
- **Komut tablosundan örnekler:**

Komut	Opcode	Mnemonic	İşlem
Move A	0000000	MOVA	$R[DR] \leftarrow R[SA]$
Increment	0000001	INC	$R[DR] \leftarrow R[SA] + 1$
Add	0000010	ADD	$R[DR] \leftarrow R[SA] + R[SB]$
Subtract	0000101	SUB	$R[DR] \leftarrow R[SA] - R[SB]$
Load Immediate	1001100	LDI	$R[DR] \leftarrow$ zf OP
Add Immediate	1000010	ADI	$R[DR] \leftarrow R[SA] +$ zf OP
Load	0010000	LD	$R[DR] \leftarrow M[R[SA]]$
Store	0100000	ST	$M[R[SA]] \leftarrow R[SB]$

Komut	Opcode	Mnemonic	İşlem
Branch on Zero	1100000	BRZ	$R[SA] = 0$ ise $PC \leftarrow PC +$ se AD
Branch on Negative	1100001	BRN	$R[SA] < 0$ ise $PC \leftarrow PC +$ se AD
Jump	1110000	JMP	$PC \leftarrow R[SA]$

- Ayrıca DEC, AND, OR, XOR, NOT, MOVB, SHR, SHL komutları vardır. Toplam 19 komut.
- **LD/ST'de dikkat:** Bellek adresi her zaman SA register'ında tutulur. ST'de belleğe yazılacak veri SB register'ındadır.

2. Sign extend ve dallanma adresi

- Adres alanı (AD): Adres left (3 bit, DR alanının yeri) ve adres right (3 bit, SB alanının yeri) yan yana konur → 6 bitlik işaretli sayı. SA alanı test edilen register'ı gösterir.
- PC 16 bittir. Toplama için AD de 16 bite genişletilmelidir.
- İşaret biti 1 ise (negatif) üstteki 10 bitin hepsi 1 ile doldurulur. 0 ise 0 ile doldurulur.
- Negatif değer geri dallanma, pozitif değer ileri dallanma demektir.

3. Bellek içeriğinden komut çözme örneği

- Adres 25: opcode 0000101 → SUB. DR = 1, SA = 2, SB = 3 → $R1 \leftarrow R2 - R3$.
- Adres 35: opcode 0100000 → ST. SA = 4, SB = 5 → $M[R4] \leftarrow R5$.
- Adres 45: ADI. DR = 2, SA = 7, OP = 3 → $R2 \leftarrow R7 + 3$.
- Adres 55: BRZ. SA = 6. AD = 101 ile 100 birleşir → `101100`.
  - İşaret biti 1 → negatif. İkiye tümleyeni `010100` = 20 → AD = −20.
  - $R6 = 0$ ise $PC \leftarrow PC - 20$ (20 adres geri dallanma).
- Adres 70: veri. Komut değildir.

4. Kontrol birimi ve komut çözücü

- PC'nin gösterdiği adresten komut belleğinden 16 bitlik komut okunur.
- Komut çözücü (instruction decoder) komuttan kontrol kelimesini üretir.
- Önceki kontrol kelimesine eklenen alanlar:
  - **MW:** belleğe yazma var mı.
  - **PL:** PC'ye yükleme (dallanma/atlama) var mı.
  - **JB:** 0 ise koşullu dallanma (branch), 1 ise atlama (jump).
  - **BC:** 0 ise "sıfırsa dallan", 1 ise "negatifse dallan".
- PC iki yoldan yüklenebilir: $PC+$se AD (dallanma) ya da A yolundan gelen register (JMP).
- Komutlar fonksiyon birimi işlemleri, bellek okuma/yazma, dallanma ve atlama diye gruplara ayrılır.
- Çözücü, opcode'un 15, 14, 13 ve 9. bitlerinden MB, MD, RW, MW, PL, JB, BC çıkışlarını üretir.
  - $MB = I_{15}$: tablodaki MB sütunu opcode'un 15. bitiyle aynıdır.
  - **RW örneği (derste birlikte çözüldü):** Doğruluk tablosu ($I_{15}, I_{14}, I_{13}, I_9$) Karnaugh haritasına taşındı.
    - 000X ve 001X için 1, 010X için 0, 100X için 1, 1100, 1101 ve 111X için 0. Tabloda olmayan gözler don't care.
    - Üst ve alt satırlar birleştirilince 8'li grup çıkar: $RW = \overline{I_{14}}$.
  - Hoca diğer çıkışları da tablodan bulup devredeki kapılarla karşılaştırmanızı tavsiye etti.
- Bu yapı hardwired kontroldür. RISC işlemcilerde kullanılır ve daha hızlıdır.

5. Tek çevrimli bilgisayarın çevrim süresi

- Bir komutun bütün adımları tek bir clock periyodunda tamamlanmalıdır.

- Gecikmeler: PC 0.2 ns, komut belleği 4 ns, register file (okuma) 0.6 ns, MUX B 0.2 ns, fonksiyon birimi veya veri belleği 4 ns, MUX D 0.2 ns, register file (yazma) 0.6 ns.
- Toplam: 9.8 ns. En kısa clock periyodu 9.8 ns olabilir.
- Clock periyodu işlemcinin frekansını belirler.

6. Pipeline datapath (3 aşama)

- Geleneksel (conventional) datapath gecikmeleri: 0.6 + 0.6 + 0.2 + 0.8 + 0.2 = 2.4 ns. $T = 2.4$ ns.
- Aşamaların arasına pipeline register'ları eklenir:
  - **OF (operand fetch):** register okuma + MUX B.
  - **EX (execution):** fonksiyon birimi.
  - **WB (write back):** MUX D + register'a yazma.
- Aşamaların süreleri farklıdır. Sistem senkron çalışmalıdır. En yavaş aşama clock periyodunu belirler: $T = 1$ ns.
- 7 mikro işlem:
  - Çevrim 1: 1. işlem OF.
  - Çevrim 2: 1. işlem EX, 2. işlem OF.
  - Çevrim 3: 1. işlem WB, 2. işlem EX, 3. işlem OF. Pipeline doldu, 1. işlem bitti.
  - 7 işlem 9 çevrimde biter: $9 \times 1 = 9$ ns.
- Geleneksel yöntem: $7 \times 2.4 = 16.8$ ns. Pipeline neredeyse 2 kat hızlıdır.
- Bir öğrenci: "Aşama sayısını artırarak daha da hızlandırabiliriz." Hoca: kontrol aşamaları da eklenince aşama sayısı artıyor.

7. Pipeline kontrol (4 aşama)

- Aşamalar:
  - **IF (instruction fetch):** komut belleğinden komutu getirme.
  - **DOF (decode and operand fetch):** komutu çözme ve operandları getirme.
  - **EX (execution):** işlemi yapma.
  - **WB (write back):** sonucu register'a ya da belleğe yazma.
- Her aşamanın sonunda bir register vardır. Sonraki aşama bu register'dan okur.
- Aşama gecikmeleri en çok 1 ns olacak şekilde bölünmüştür → en çok 1 GHz.
- Örnek: R1-R7 register'larına sırasıyla 1-7 sabitlerini yükleyen 7 komut.
- Toplam çevrim = (aşama sayısı − 1) + komut sayısı = $(4 - 1) + 7 = 10$.
  - Pipeline: $10 \times 1 = 10$ ns.
  - Geleneksel (tek çevrim 3.4 ns): $7 \times 3.4 = 23.8$ ns.
  - Hızlanma: $23.8/10 \approx 2.4$ kat.

8. Pipeline tehlikeleri (hazards)

- **Pipeline'ı doldurma ve boşaltma:** Pipeline dolana kadar çıktı yoktur. Dallanmada pipeline boşaltılmalıdır. Bu performansı düşürür.
- **Veri bağımlılığı (data hazard):**
  - Örnek: `MOVA R1, R5` → `ADD R2, R1, R6` → `ADD R3, R1, R2`.
  - 1. komut R1'e 4. çevrimde (WB) yazar. 2. komut R1'i 3. çevrimde (DOF) okumak ister. Değer henüz yazılmamıştır.
  - **Yazılım çözümü:** araya NOP (no operation) eklenir. Böylece yazma ve okuma aynı çevrime denk gelir. Aynı problem R2 için de vardır.
  - **Donanım çözümü:** tehlike algılanınca pipeline bekletilir (stall, "bubble") ya da EX sonucu doğrudan sonraki komuta iletilir.
  - Bir öğrenci sordu: "Bir NOP her zaman yeter mi?" Hoca: bu 4 aşamalı yapıda yeter. Aşama sayısı fazla olan mimarilerde yetmeyebilir.
  - NOP eklemek pipeline'ın verimini düşürür. Ama tedbir alınmazsa işlemci yanlış çalışır.
- **Dallanma tehlikesi (branch hazard):**
  - Pipeline her zaman bir sonraki satırın çalışacağını varsayar.
  - Dallanma gerçekleştiğinde arkadan pipeline'a giren komutlar yanlıştır ve silinmelidir.
  - Çözüm: dallanma komutundan sonra 2 NOP eklenir.

- Komutun dallanma olduğu IF aşamasında okunan opcode'dan anlaşılır.

9. Giriş/Çıkış birimleri ve G/Ç arayüzü

- Çevre birimleri (peripherals): klavye, ekran, yazıcı, manyetik disk, tarayıcı, modem, hoparlör, mikrofon.
  - Analog birimler için ADC/DAC dönüşümü gerekir. Bu dersin konusu değildir.
- **Klavye:** tuşlar bir tarama matrisi (scan matrix) üzerindedir. Satırlar decoder ile sürülür, sütunlar MUX ile okunur. Klavyedeki mikrodenetleyici basılan tuşun satır/sütun bilgisinden tarama kodu (scan code) üretir.
- **Neden arayüz gerekir?** Çevre birimleri çoğunlukla elektromekaniktir. Farklı gerilim seviyeleri (ör. 0-12 V) ve farklı veri formatları kullanabilirler.
- Veri yolu çift yönlüdür. Adres yolu tek yönlüdür (CPU'dan dışarı). Kontrol yolundaki işaretlerin bazıları giriş, bazıları çıkıştır.
- Giriş birimi arayüzü veri yoluna yalnızca giriş yönünde, çıkış birimi yalnızca çıkış yönünde, disk ise çift yönlü bağlanır.
- **Port erişim yöntemleri:**
  - **Isolated I/O:** adres ve veri yolu ortak, kontrol hatları farklı. Özel IN/OUT komutları vardır. Intel 8086 bunu kullanır.
  - **Memory-mapped I/O:** bellek ve G/Ç için ortak adres, veri ve kontrol yolu. Portlar bellek adresi gibi kullanılır. Motorola işlemcilerde vardır.

10. Örnek G/Ç arayüzünün iç yapısı

- 4 iç register: Port A, Port B, Control, Status. Sayısı arayüze göre değişir.
- Sol taraftaki uçlar CPU'ya, sağ taraftakiler G/Ç birimine bağlanır.
- Timing and control bloğu chip select (CS) ve register select (RS1, RS0) uçlarına bakar:

CS	RS1	RS0	Seçilen
0	X	X	Hiçbiri (veri yolu high-Z)
1	0	0	Port A
1	0	1	Port B
1	1	0	Control register
1	1	1	Status register

- **Adres çözme örneği:** Arayüz G/Ç uzayında A8h-ABh adreslerine yerleştirilsin (4 register → 4 adres).
  - A8h = `1010 1000`, ABh = `1010 1011`. Yalnızca A1 ve A0 değişir. Bunlar RS1 ve RS0'a bağlanır.
  - Sabit kalan A7-A2 = `101010`. CS için 6 girişli AND kapısı: A7, A5, A3 doğrudan, A6, A4, A2 NOT kapısından geçerek bağlanır.
  - `IN AL, A8h` → Port A okunur. Port B (A9h) klavyenin tarama kodunu tutuyorsa `IN AL, A9h` basılan tuşun kodunu AL'ye okur.

11. Arayüz ile cihaz arasındaki haberleşme

- **Strobing (tek tel):**
  - Destination-initiated: hedef birim önce strobe işareti üretir, kaynak veriyi koyar.
  - Source-initiated: kaynak (ör. klavye) veriyi veri yoluna koyar ve strobe gönderir. Hedef veriyi okur.
  - Sorun: karşı tarafın veriyi okuyup okumadığına dair geri bildirim yoktur.
- **Handshaking (iki tel):**
  - Hedef birim request gönderir.
  - Kaynak veriyi veri yoluna koyar ve reply işaretini verir.
  - Hedef veriyi okuduktan sonra request'i kaldırır. Böylece kaynak verinin alındığını anlar.
  - Daha güvenli bir iletişimdir.

12. Paralel ve seri iletişim

- **Paralel:** n bit aynı anda n telden gönderilir. Daha hızlıdır.
- **Seri:** bitler tek tel üzerinden arka arkaya gönderilir. Daha yavaş ama daha ekonomiktir. Modem ve USB örnek.
- Seri iletişim modları:
  - **Simplex:** tek yön (radyo yayını).
  - **Half duplex:** iki yön, ama aynı anda tek yön (telsiz).
  - **Full duplex:** aynı anda iki yön.
- Asenkron/senkron seri haberleşme, start/stop ve parity bitleri sonraki derse kaldı.

Ders sonrası notlar

- **Çevrim süresi hesabı:** Hoca toplamı ekranda yüksek sesle toplarken "9 oldu, 9.8 mi oldu?" diye duraksadı. Doğrusu $0.2 + 4 + 0.6 + 0.2 + 4 + 0.2 + 0.6 = 9.8$ ns'dir.
- **İki farklı tek çevrim süresi:** 9.8 ns, kitaptaki 4 ns'lik bellek gecikmeleriyle hesaplanan örnektir. Pipeline karşılaştırmasında ise slayttaki varsayım tek çevrimli bilgisayar için 3.4 ns'dir ("1 GHz, tek çevrimlinin 3.4 katı"). İki rakamı karıştırmayın.
- **3 aşamalı hızlanma:** $16.8/9 \approx 1.87$. Hoca "neredeyse iki kat" dedi, doğru.
- **4 aşamalı hızlanma:** $23.8/10 = 2.38$. Hoca "yaklaşık 2.4 kat" dedi, doğru.
- **Dallanma ne zaman anlaşılır?** Hoca önce "birinci komut write back'e geldiğinde dallanma olduğunu anlıyorsunuz" dedi. Dallanma kararı WB'de verilseydi arkadan 3 komut girmiş olurdu ve 3 NOP gerekirdi. 2 NOP yetmesi, kararın EX aşamasında verildiğini gösterir. O anda IF ve DOF aşamalarındaki iki komut silinmelidir.
- **NOP'u kim ekler?** Hoca bir yerde "işlemci içinde optimizer", bir yerde "işletim sistemi arka tarafta çalışan bir optimizer" dedi. Yazılım çözümünde NOP'ları derleyici (compiler) ya da assembler program yazılırken ekler. İşletim sistemi çalışma anında NOP eklemez. Hoca da sonraki derste bunun "compiler düzeyinde" çözülebileceğini söyledi.
- **"Software kontrol":** Hardwired kontrolün karşılaştırıldığı yapının adı mikroprogramlı kontroldür (microprogrammed control).
- **Ekran karesinden alınan donanım çözümü:** Slaytta veri tehlikesi algılanınca pipeline'ın bekletildiği (stall) ve "bubble" gönderildiği gösteriliyor. Hoca bunu sözlü olarak "sonucu doğrudan aktaran blok" diye özetledi. İki yaklaşım da (stall ve forwarding) donanım çözümüdür.
- **Dallanma adresinin alanları:** Hoca bir yerde "source A adresi ve source B adresi … toplanıyor" dedi. Doğrusu adres left ve adres right alanları birleştirilir, toplanmaz. SA test edilen register'dır. Adres 55 örneğinde de SA = 6, AD = `101` ile `100`'dür.
- **RW haritası:** Hoca tabloda yer almayan "iki tane göz" dedi. Tabloda 011X ve 101X satırları yoktur. Yani 4 göz don't care'dir.
- **Klavye:** Hoca "matrisin sütunlarını süren bir multiplexer ve bir decoder" dedi. Ders 11'de düzeltti: satırları decoder sürer, sütunlardan gelen bilgiyi MUX toplar.
- **Örnek arayüz:** Hoca iç register sayısı için "iki tane olan bir yapı" dedi. Örnek arayüzde 4 iç register vardır.
- **Strobing:** Hoca "iki telli haberleşme, buna strobing deniyor" dedi. Strobing tek tellidir. İki telli olan handshaking'dir.
- Altyazıda "Bilseyer donanımı" Bilgisayar Donanımı, "end shaking" handshaking, "TOOS Komplement" two's complement demektir.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.